

MATH 231 – Linear Algebra I

The Computational Unit, Part 2 ■ k -Nearest Neighbours

Kiely Hall 326 ■ Instructor: Kevin Bedoya

About this document. This is Part 2 of the computational unit, and the first of two on algorithms that do nothing but measure distances between vectors. Everything it needs was built in the vector unit; what is new is putting those tools to work on a problem, and then asking what the work costs.

Part	Sections	Subject
1	§1–§4	operation counting, Big-O, floating point, error
2	§5–§10	k -nearest neighbours
3	§11–§15	k -means clustering

A common confusion (Whose §5 is this?). Section numbering runs continuously through the computational unit, so a bare “§1.4” below means §1.4 *of this unit* — Part 1. The vector unit and the matrix unit have their own numbering, and references into them always say so, as in “the vector unit, §14.2”.

Assumed. From the vector unit: $\mathbb{R}^n$ and its arithmetic (§24); the Euclidean norm (§2) and distance $d(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|$ (§14.2); the metric axioms **M1–M4** (§14.1); the family of p -norms and the Manhattan and Chebyshev metrics (§14.3–§14.6); cosine similarity (§11.4). From Part 1 of this unit: what an operation is (§1.2), the cost of a norm (§1.4), Big-O (§2.7), and FLOPs (§4.6).

What you should be able to do afterwards. State the classification problem in vector terms; describe the k -nearest neighbours rule and run it by hand; write its pseudocode; count its operations and give its complexity; explain what changes as k grows and why feature scaling matters; distinguish k -NN from clustering and build a k -NN graph; and name the standard ways the algorithm is made fast in practice.

5. Data as vectors

5.1. Turning a thing into a point

Everything in this part rests on one move, and the move is so routine in practice that it is easy to miss how much it assumes. Take some collection of objects — patients, photographs, emails, houses for sale — and describe each one by a fixed list of n numbers. That list is a vector in $\mathbb{R}^n$, and the object has become a point.

Definition 5.1 (Feature vector). A **feature vector** is a vector $\mathbf{x} = [x_1, \dots, x_n] \in \mathbb{R}^n$ whose components are n measured attributes of a single object, recorded in a fixed order. Each component is a **feature**, and n is the **dimension** of the data.

A house might become [area, bedrooms, age, distance to station] $\in \mathbb{R}^4$. A grayscale image 28×28 pixels becomes the list of its 784 brightness values, a point in $\mathbb{R}^{784}$. A sentence, run through a modern language model, becomes a vector of a few hundred or a few thousand numbers.

Definition 5.2 (Dataset). A **labelled dataset** is a finite collection

$$\mathcal{D} = \{(\mathbf{x}_1, y_1), (\mathbf{x}_2, y_2), \dots, (\mathbf{x}_N, y_N)\}, \quad \mathbf{x}_i \in \mathbb{R}^n,$$

of N feature vectors, each carrying a **label** y_i . When the labels come from a finite set of categories the problem is **classification**; when they are real numbers it is **regression**.

Two numbers will recur throughout, and they measure different things: N is how *many* points there are, and n is how *long* each one is. Confusing them is the most common error in a first complexity analysis.

Remark (What the move assumes). Writing an object as a point in $\mathbb{R}^n$ commits to something: that the arithmetic of $\mathbb{R}^n$ means something about the objects. Specifically, it commits to *distance being meaningful* — to the claim that two houses whose feature vectors are close really are similar houses. That is a modelling assumption, not a mathematical fact, and §8.2 is about a way it commonly fails.

5.2. The classification problem

Given a labelled dataset $\mathcal{D}$ and a new, unlabelled point $\mathbf{q} \in \mathbb{R}^n$ — the **query** — predict its label.

The dataset is what we have been told; the query is what we are asked about. A **classifier** is any rule that turns the first into an answer about the second.

5.3. The idea in one sentence

Here is the whole of k -nearest neighbours, before any notation:

To label a new point, look at the labelled points nearest to it, and let them vote.

That is an idea, not yet an algorithm, and turning it into one requires answering exactly three questions.

Q1. What does *nearest* mean? — §5.4.

Q2. How many neighbours vote? — that is the k , §6.2.

Q3. How do they vote? — §6.3.

5.4. What “nearest” means

From the vector unit. The vector unit answered Q1 in advance. A *metric* is a function $d(\mathbf{x}, \mathbf{y})$ measuring distance, subject to the four axioms **M1–M4** of §14.1, and the unit built a whole family of them. The default is Euclidean distance,

$$d_2(\mathbf{x}, \mathbf{y}) = \|\mathbf{x} - \mathbf{y}\|_2 = \sqrt{\sum_{i=1}^n (x_i - y_i)^2} \quad (\S 14.2),$$

but nothing in the algorithm requires it. The Manhattan distance d_1 (§14.3), the Chebyshev distance d_∞ (§14.4), and any p -norm distance (§14.6) all satisfy the axioms and all may be substituted directly.

Which one to use is a modelling decision. Manhattan distance is natural when the features are independent counts and you want each to contribute in proportion to its own deviation; Euclidean is natural when the features are spatial coordinates. For text and for high-dimensional embeddings, practitioners very often use *cosine* distance instead,

$$d_{\cos}(\mathbf{x}, \mathbf{y}) = 1 - \cos(\mathbf{x}, \mathbf{y}) = 1 - \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{\|\mathbf{x}\| \|\mathbf{y}\|} \quad (\text{the vector unit, §11.4}),$$

which compares *direction* and ignores magnitude entirely — appropriate when a long document and a short one on the same topic should count as similar.

Remark. $d_{\cos}$ is not a metric: it fails the triangle inequality **M4**. This is worth knowing and, for k -NN, worth not worrying about — the algorithm only ever *sorts* by the distance function, and sorting needs an ordering, not an axiom. Some of the fast methods in §10.2 do rely on the triangle inequality, and those are the ones that stop working.

6. The algorithm

6.1. Walking through it

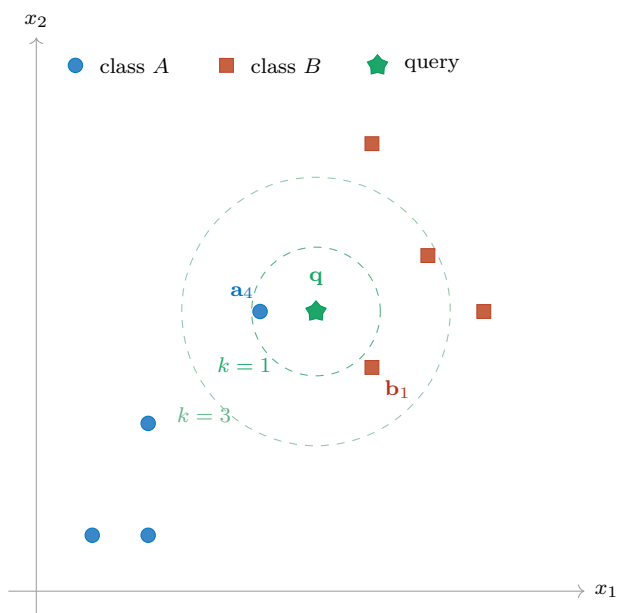
Fix a metric d and an integer $k \geq 1$. Given the query $\mathbf{q}$:

Step 1. Measure. Compute the distance from $\mathbf{q}$ to every one of the N points in $\mathcal{D}$. This produces N numbers $d(\mathbf{q}, \mathbf{x}_1), \dots, d(\mathbf{q}, \mathbf{x}_N)$ and nothing else.

Step 2. Select. Find the k smallest of those numbers, and remember which points they belonged to. Call that set of points the k -**neighbourhood** of $\mathbf{q}$, written $\mathcal{N}_k(\mathbf{q})$.

Step 3. Vote. Look at the labels of those k points. Predict whichever label occurs most often.

That is the entire algorithm. Notice what is *not* in it: there is no model, no equation fitted to the data, no parameters estimated. The dataset is the model. Everything happens when a query arrives.



The two dashed circles are the picture of Step 2: growing a ball around $\mathbf{q}$ until it contains exactly k points. The inner circle catches one point, which is blue; the outer catches three, of which two are red. So the two settings of k disagree, and §6.5 works out why in numbers.

6.2. The k

Definition 6.1 (k -neighbourhood). The k -**neighbourhood** of a query $\mathbf{q}$, written $\mathcal{N}_k(\mathbf{q})$, is the set of the k points of the dataset $\mathcal{D}$ lying closest to $\mathbf{q}$ under the metric d .

Two extreme settings frame the choice.

- $k = 1$ is the **nearest neighbour** rule: copy the label of the single closest point. It follows the data exactly — and therefore follows its mistakes exactly too. One mislabelled or unusual training point creates a small region around itself where every query gets the wrong answer.
- $k = N$ makes every query return the most common label in the entire dataset, regardless of where the query is. The distances have stopped mattering.

Useful values live between, and §8.1 says how to think about choosing. For a two-class problem it is customary to take k *odd*, which makes a tied vote impossible.

6.3. Pseudocode

Write $\mathbf{1}[P]$ for the *indicator* of a statement P : it is 1 when P is true and 0 when P is false. Then counting how many neighbours carry the label c is a sum of indicators, and the vote is an $\arg \max$ over the possible labels.

k -NN classification

Input: a labelled dataset $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$ with $\mathbf{x}_i \in \mathbb{R}^n$; a query $\mathbf{q} \in \mathbb{R}^n$; a metric d ; an integer $k \geq 1$. *Output:* a predicted label $\hat{y}$.

```

1: for  $i = 1$  to  $N$  do Step 1: measure
2:    $\delta_i \leftarrow d(\mathbf{q}, \mathbf{x}_i)$ 
3: end for
4: let  $\pi$  be a permutation of  $\{1, \dots, N\}$  with  $\delta_{\pi(1)} \leq \delta_{\pi(2)} \leq \dots \leq \delta_{\pi(N)}$  Step 2: select
5:  $\mathcal{N}_k(\mathbf{q}) \leftarrow \{\pi(1), \pi(2), \dots, \pi(k)\}$ 
6: for each label  $c$  occurring in  $\mathcal{D}$  do Step 3: vote
7:    $\text{votes}(c) \leftarrow \sum_{i \in \mathcal{N}_k(\mathbf{q})} \mathbf{1}[y_i = c]$ 
8: end for
9:  $\hat{y} \leftarrow \arg \max_c \text{votes}(c)$ 
10: return  $\hat{y}$ 

```

Two lines deserve a second look.

Line 4 asks for a permutation that sorts the distances, which is more than the algorithm needs — only the first k entries of π are ever used. Line 4 is written this way because it states *what* is wanted with no commitment to *how*, and §7.2 shows the choice of how is worth a factor of $\log N$.

Line 9 hides a decision. If two labels tie for the maximum, $\arg \max$ does not say which to return. §8.4 supplies the usual conventions.

Remark (The regression variant). If the labels y_i are real numbers rather than categories, the vote in lines 6–9 is replaced by an average:

$$\hat{y} = \frac{1}{k} \sum_{i \in \mathcal{N}_k(\mathbf{q})} y_i.$$

Nothing else changes. Steps 1 and 2 — which are the expensive ones — are identical.

6.4. A refinement: weighting by distance

The vote of §6.3 treats all k neighbours equally, so a point on the far edge of the neighbourhood counts as much as one sitting almost on top of the query. A common fix weights each vote by $1/\delta_i$, or by $1/\delta_i^2$:

$$\text{votes}(c) = \sum_{i \in \mathcal{N}_k(\mathbf{q})} \frac{\mathbf{1}[y_i = c]}{\delta_i}.$$

This costs k extra divisions and nothing else, so it does not change any complexity in §7. It also softens the choice of k : distant neighbours are admitted to the vote but do not count for much, so the answer changes less abruptly as k grows.

6.5. A worked example

Example 6.1. Work in $\mathbb{R}^2$ with Euclidean distance. The dataset is eight points, four in class A and four in class B :

$$\begin{aligned} A: \quad \mathbf{a}_1 &= [1, 1], \quad \mathbf{a}_2 = [2, 3], \quad \mathbf{a}_3 = [2, 1], \quad \mathbf{a}_4 = [4, 5], \\ B: \quad \mathbf{b}_1 &= [6, 4], \quad \mathbf{b}_2 = [7, 6], \quad \mathbf{b}_3 = [6, 8], \quad \mathbf{b}_4 = [8, 5]. \end{aligned}$$

The query is $\mathbf{q} = [5, 5]$. Step 1 computes eight distances. It is cheaper and equivalent to work with δ^2 — see §7.5 — so both are tabulated:

Point	Label	δ^2	δ	Rank
$\mathbf{a}_4 = [4, 5]$	A	1	1.000	1
$\mathbf{b}_1 = [6, 4]$	B	2	1.414	2
$\mathbf{b}_2 = [7, 6]$	B	5	2.236	3
$\mathbf{b}_4 = [8, 5]$	B	9	3.000	4
$\mathbf{b}_3 = [6, 8]$	B	10	3.162	5
$\mathbf{a}_2 = [2, 3]$	A	13	3.606	6
$\mathbf{a}_3 = [2, 1]$	A	25	5.000	7
$\mathbf{a}_1 = [1, 1]$	A	32	5.657	8

All eight distances are distinct, so Step 2 has no ambiguity. Now vary k :

k	$\mathcal{N}_k(\mathbf{q})$	Votes	Prediction
1	$\mathbf{a}_4$	$A : 1, B : 0$	$\hat{y} = A$
3	$\mathbf{a}_4, \mathbf{b}_1, \mathbf{b}_2$	$A : 1, B : 2$	$\hat{y} = B$
5	$\mathbf{a}_4, \mathbf{b}_1, \mathbf{b}_2, \mathbf{b}_4, \mathbf{b}_3$	$A : 1, B : 4$	$\hat{y} = B$

The answer at $k = 1$ disagrees with the answer at $k = 3$ and $k = 5$, and the table shows exactly why. The point $\mathbf{a}_4 = [4, 5]$ carries label A but sits deep inside the region occupied by B — an outlier, or a mislabelling, or a genuinely unusual case. Being nearest to $\mathbf{q}$, it decides the vote by itself when $k = 1$. As soon as two more neighbours are admitted they are both B , and they outvote it. ■

Remark. This example is the argument for $k > 1$ in miniature. Raising k does not make the classifier smarter; it makes it *less sensitive to any single training point*. That is worth having when the data contains noise, and it is exactly what you do not want when the data is clean and the true boundary is intricate. §8.1 is that trade-off, stated.

7. What it costs

Now the question Part 1 taught us to ask. We count the work for *one* query against a dataset of N points in $\mathbb{R}^n$, in FLOPs (§4.6).

7.1. Step 1: the distances

One Euclidean distance $d(\mathbf{q}, \mathbf{x}_i) = \|\mathbf{q} - \mathbf{x}_i\|$ is two operations we have already priced. Forming the difference $\mathbf{q} - \mathbf{x}_i$ is vector subtraction, n FLOPs by §1.1. Taking its norm is $2n$ FLOPs by §1.4 — n mults to square, $n - 1$ adds to total, one square root. So

$$\text{one distance} = n + 2n = 3n \text{ FLOPs,}$$

and Step 1 performs N of them:

$\text{Step 1} = 3Nn \text{ FLOPs} = O(Nn).$

Remark. The count is linear in *both* inputs, and both matter. Doubling the number of training points doubles the work; so does doubling the number of features. This is the sense in which k -NN has no free lunch: every training point is touched, every time, at full length.

7.2. Step 2: the selection

Step 2 must find the k smallest of N numbers. There are three standard ways, and they are not equally good.

Method	Cost	Comment
Sort all N distances	$O(N \log N)$	what line 4 literally says; wasteful
Keep a heap of the k best	$O(N \log k)$	one pass, k items retained
Quickselect	$O(N)$ average	$O(N^2)$ worst case

The middle row is what libraries do. Scan the N distances once, maintaining a structure holding the best k seen so far; each new distance costs at most $O(\log k)$ to insert or reject. Since k is typically a small constant — 3, 5, 11 — and N may be in the millions, the difference between $\log k$ and $\log N$ is the difference between a factor of 2 and a factor of 20.

7.3. Step 3: the vote

Counting the labels of k points and taking the largest count is $O(k)$, and $k \leq N$. Beside Steps 1 and 2 this is free, and it will not appear again.

7.4. The total

Adding the three steps,

$$T(N, n, k) = \underbrace{O(Nn)}_{\text{distances}} + \underbrace{O(N \log k)}_{\text{selection}} + \underbrace{O(k)}_{\text{vote}} = \boxed{O(Nn + N \log k)}$$

per query. In almost every real setting the first term dominates: n is the number of features, often hundreds or thousands, while $\log k$ is a small number like 2 or 3. So the working summary is that a k -NN query costs $O(Nn)$, and M queries cost $O(MNn)$.

Quantity	Cost	Why
Training	$O(1)$	store the data; nothing is computed
Memory	$O(Nn)$	every training point must be kept
One query	$O(Nn + N \log k)$	§7.4
M queries	$O(MNn)$	no work is shared between queries

Remark (A lazy learner). The first row is the strangest entry in the table and the most characteristic. Training a k -NN classifier costs nothing: you keep the data. Algorithms that behave this way are called **lazy** — they defer all computation to query time — as opposed to **eager** learners, which do expensive work once up front and then answer queries cheaply.

Neither is better in the abstract, and the choice is a choice about where you can afford to spend. Laziness is excellent when the dataset changes constantly, because there is no model to rebuild; adding a training point is free. It is terrible when queries are frequent and the dataset is large, because you pay $O(Nn)$ every single time and never amortise it. §10.2 is about buying back some of that cost with a one-off index.

7.5. One saving, and it is Part 1's saving

There is a cheap improvement to Step 1 that costs nothing to adopt. The algorithm never uses the *value* of a distance — only the *order* of the distances. And squaring is increasing on non-negative numbers, so for $\delta, \delta' \geq 0$,

$$\delta \leq \delta' \iff \delta^2 \leq \delta'^2.$$

The ranking by δ and the ranking by δ^2 are therefore the same ranking, and we may compare squared distances and never take a square root:

$$\delta_i^2 = \sum_{j=1}^n (q_j - x_{ij})^2 \quad \text{costs } n + n + (n - 1) = 3n - 1 \text{ FLOPs.}$$

The saving is one square root per training point, so N square roots per query.

Remark. This is exactly the second saving of §1.6 — “do not take a square root you are going to square” — arriving in a new setting, and it is worth noticing that the argument is not the same one. In §1.6 the square root cancelled algebraically. Here it does not cancel at all; it is discarded because the answer only depends on the ordering, and the square root preserves orderings. Different reason, same square root, same saving.

In Big-O terms nothing has happened: $3n - 1$ and $3n$ are both $O(n)$. In wall clock terms something has, because a square root is among the slowest operations a processor offers (§1.2), and this removes one per training point per query. It is the standard implementation, and the table in §6.5 was built on it.

8. Choosing k , and what goes wrong

8.1. Small k , large k

The example of §6.5 showed the two failure directions in one dataset.

	k too small	k too large
Boundary shape	jagged, follows every point	smooth, ignores real detail
Sensitive to	noise and mislabelling	nothing much
Failure mode	memorises the training data	predicts the majority label
Standard name	<i>overfitting</i>	<i>underfitting</i>

There is no formula for the right k . It is chosen empirically: hold back part of the labelled data, run the classifier on it for several values of k , and keep the value that gets the most answers right. Doing this properly — so that the held-back data is not quietly used to make the choice it is supposed to be judging — is called *cross-validation*, and it belongs to a statistics course rather than this one. A common starting guess is $k \approx \sqrt{N}$.

8.2. Feature scaling, and why it is not optional

This is the failure that catches everyone once. Suppose houses are described by

$$\mathbf{x} = [\underbrace{\text{area in m}^2}_{\text{roughly 50–300}}, \underbrace{\text{number of bedrooms}}_{\text{roughly 1–5}}].$$

Take two houses of equal area differing by 4 bedrooms, and two houses with the same bedrooms differing by 4 square metres. Squared Euclidean distance scores both differences as 16. But one pair is a studio next to a family home and the other pair is indistinguishable.

The arithmetic is doing precisely what it was told. Euclidean distance adds the squared differences of the components with equal weight, so a feature measured in large units contributes large differences and dominates the sum. Change the area to square centimetres and it will dominate absolutely; change it to hectares and it will vanish. *The units of your features silently set their importance.*

Definition 8.1 (Standardization). To **standardize** a dataset, replace each feature by

$$x_{ij} \mapsto \frac{x_{ij} - \mu_j}{\sigma_j},$$

where μ_j is the mean of feature j across all N points and σ_j is its standard deviation. Every feature then has mean 0 and spread 1, and none is privileged by its units.

A common confusion (Distance is not unit-free). Nothing in the metric axioms of the vector unit, §14.1, protects you here. d_2 is a perfectly good metric on $\mathbb{R}^n$ whatever the columns mean; the axioms constrain how d behaves, not whether the numbers it is fed are comparable. Making them comparable is your job, it happens before the algorithm runs, and skipping it does not produce an error — it produces a classifier that quietly ignores most of your features.

Remark. An alternative to rescaling is to change the metric. Instead of weighting every feature equally, use $d(\mathbf{x}, \mathbf{y})^2 = \sum_j w_j (x_j - y_j)^2$ with weights w_j chosen to reflect how much each feature should matter. Standardization is the special case $w_j = 1/\sigma_j^2$, which says “every feature matters equally once its own spread is accounted for”.

8.3. The curse of dimensionality

The third difficulty is subtler and it is specific to large n . As the dimension grows, the distances from a query to all the training points tend to become nearly equal — the nearest point is barely nearer than the farthest. When that happens the phrase “the k nearest neighbours” stops picking out anything meaningful: the ranking still exists, but it is decided by noise.

The intuition is a volume count. In $\mathbb{R}^n$ a ball of radius r has volume proportional to r^n , so a small increase in radius sweeps up a huge additional region. Points spread through a high-dimensional space are overwhelmingly likely to sit at similar middling distances from each other, and close neighbours become vanishingly rare unless N grows exponentially in n .

Remark. This is not an argument against k -NN, which remains one of the most useful methods there is on high-dimensional data — image and text retrieval are exactly that. It is an argument for not using the *raw* dimension. The standard response is to reduce the dimension first, mapping the data into a space of far fewer coordinates that preserves the distances that matter, and then run k -NN there. The tool for doing that linearly is the singular value decomposition, which is a later unit of this course.

8.4. Ties

Two kinds of tie can occur, and both need a rule stated in advance.

- **A tie in distance** at the boundary of the neighbourhood: more than k points are within the cutoff. Conventions: take all of them; take the ones with the lowest index; or break the tie at random. Any is defensible, and libraries differ, so it must be checked rather than assumed.
- **A tie in the vote**: two labels receive the same count. The usual rules are to fall back to the label of the single nearest neighbour, to decrease k by one and re-vote, or — best — to use distance weighting (§6.4), under which exact ties are essentially impossible. Choosing k odd removes the problem for two classes and does nothing for three.

Remark. It is worth remembering, from Part 1, that these are floating-point distances (§3). Two distances that are equal in exact arithmetic may differ in the last bit after rounding, and two that differ mathematically may round to the same float. So a tie-breaking rule is not only a modelling convention; it is also what keeps the output from depending on rounding.

9. k -NN and clustering

9.1. Two different problems

A *common confusion* (k -NN does not cluster). The phrase “ k -NN clustering” is in wide circulation and it conflates two genuinely different problems.

k -NN is **supervised**. It requires a labelled dataset, and it answers a question about one new point: what label should $\mathbf{q}$ get? It never partitions anything.

Clustering is **unsupervised**. There are no labels. The task is to partition the dataset itself into groups of mutually similar points, and the groups are the output. That is Part 3’s subject, k -means.

The two are neighbours in every syllabus because both run on nothing but distances between feature vectors, and the k in each is an integer the user must choose — but they are different k ’s answering different questions.

There is nevertheless a real construction that the phrase legitimately names, and it is worth having, because it is the substrate of several genuine clustering algorithms.

9.2. The k -NN graph

Definition 9.1 (k -nearest-neighbour graph). Let $P = \{\mathbf{p}_1, \dots, \mathbf{p}_N\} \subseteq \mathbb{R}^n$ and fix a metric d and an integer k . The k -NN **graph** of P has the points of P as its vertices, and joins $\mathbf{p}_i$ to $\mathbf{p}_j$ whenever $\mathbf{p}_j$ is one of the k nearest neighbours of $\mathbf{p}_i$, or $\mathbf{p}_i$ is one of the k nearest neighbours of $\mathbf{p}_j$.

The construction is k -NN run N times, once with each point of P as the query and the rest as the dataset — so it costs $O(N^2n)$ by §7.4, with no labels involved anywhere. What comes out is not a classification but a *structure*: which points are near which.

Now cluster the graph rather than the points. The simplest rule is to take the **connected components** — the groups of vertices reachable from one another by following edges — and call each one a cluster. Two points end up together when a chain of short hops connects them, even if they are far apart directly, which is the essential difference from k -means and is why this family can find long, curved clusters that k -means cannot.

9.3. A worked example

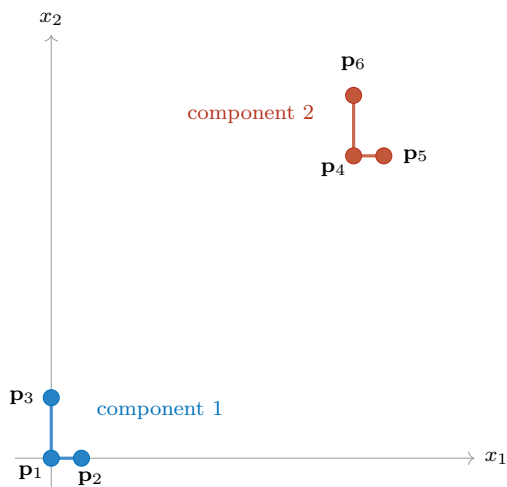
Example 9.1. Six unlabelled points in $\mathbb{R}^2$:

$$\mathbf{p}_1 = [0, 0], \quad \mathbf{p}_2 = [1, 0], \quad \mathbf{p}_3 = [0, 2], \quad \mathbf{p}_4 = [10, 10], \quad \mathbf{p}_5 = [11, 10], \quad \mathbf{p}_6 = [10, 12].$$

Build the 1-NN graph under Euclidean distance. For each point, find its single nearest neighbour:

Point	Distances to the others	Nearest	Edge
$\mathbf{p}_1$	$d(\mathbf{p}_2) = 1, d(\mathbf{p}_3) = 2, \text{ rest } \geq 14.1$	$\mathbf{p}_2$	$\{\mathbf{p}_1, \mathbf{p}_2\}$
$\mathbf{p}_2$	$d(\mathbf{p}_1) = 1, d(\mathbf{p}_3) = \sqrt{5} \approx 2.24, \text{ rest } \geq 13.5$	$\mathbf{p}_1$	$\{\mathbf{p}_1, \mathbf{p}_2\}$
$\mathbf{p}_3$	$d(\mathbf{p}_1) = 2, d(\mathbf{p}_2) \approx 2.24, \text{ rest } \geq 12.8$	$\mathbf{p}_1$	$\{\mathbf{p}_1, \mathbf{p}_3\}$
$\mathbf{p}_4$	$d(\mathbf{p}_5) = 1, d(\mathbf{p}_6) = 2, \text{ rest } \geq 12.8$	$\mathbf{p}_5$	$\{\mathbf{p}_4, \mathbf{p}_5\}$
$\mathbf{p}_5$	$d(\mathbf{p}_4) = 1, d(\mathbf{p}_6) = \sqrt{5} \approx 2.24, \text{ rest } \geq 14.9$	$\mathbf{p}_4$	$\{\mathbf{p}_4, \mathbf{p}_5\}$
$\mathbf{p}_6$	$d(\mathbf{p}_4) = 2, d(\mathbf{p}_5) \approx 2.24, \text{ rest } \geq 15.6$	$\mathbf{p}_4$	$\{\mathbf{p}_4, \mathbf{p}_6\}$

The distinct edges are $\{\mathbf{p}_1, \mathbf{p}_2\}$, $\{\mathbf{p}_1, \mathbf{p}_3\}$, $\{\mathbf{p}_4, \mathbf{p}_5\}$, $\{\mathbf{p}_4, \mathbf{p}_6\}$, and no edge joins the two groups, because every cross distance exceeds 12 while every within-group distance is at most 2.24.



The graph has exactly two connected components, $\{\mathbf{p}_1, \mathbf{p}_2, \mathbf{p}_3\}$ and $\{\mathbf{p}_4, \mathbf{p}_5, \mathbf{p}_6\}$, and those are the two clusters. No labels were used and none were available: this is unsupervised, and it is what “ k -NN clustering” properly refers to. ■

Remark. Connected components are the crudest way to cut a k -NN graph and are shown here because they can be read off by eye. Real methods cut it more carefully: *spectral clustering* uses the eigenvectors of a matrix built from the graph — a technique this course will be able to explain once eigenvalues arrive — and *DBSCAN* and *HDBSCAN* use neighbourhood density to decide which edges to keep. *UMAP*, the standard tool for visualising high-dimensional data, begins by building exactly this graph. All of them start where §9.2 starts.

10. In practice

10.1. Where it is used

Application	The vectors, and the metric
Recommendation	Each user is a vector of ratings; neighbours are users with similar taste, usually under cosine distance, and their preferences are the prediction.
Image retrieval	An image becomes a vector via a neural network; “find similar images” is literally a nearest-neighbour query.
Semantic search, RAG	Text passages are embedded as vectors; a question is embedded the same way, and the k nearest passages are retrieved and handed to a language model. This is now among the largest uses of nearest-neighbour search anywhere.
Anomaly detection	The distance to the k -th nearest neighbour is an outlier score: points in sparse regions are far from everything.
Handwriting recognition	The classic benchmark. Plain k -NN with Euclidean distance on raw 28×28 pixel vectors classifies handwritten digits correctly about 97% of the time — with no training at all, which is why it remains the baseline every new method is expected to beat.

The pattern across the table is worth naming. k -NN is chosen when similarity is the whole question, when the data changes often enough that retraining a model would be a nuisance, and when an answer needs to be explainable — “we predicted this because of those five specific cases” is a justification a person can check.

10.2. Making it fast

Everything in §7 assumed the obvious implementation: compare the query against all N points. That is called **brute force** or exhaustive search, and its $O(Nn)$ per query is the thing every optimisation attacks. The methods divide into two families by whether they still return the true answer.

Exact methods

These return exactly what §6.3 would, faster, by ruling out regions of space without examining the points in them.

- **k -d trees.** Recursively split the data by one coordinate at a time, building a binary tree. A query descends the tree and prunes any branch whose region is too far away to contain a neighbour. Queries cost about $O(\log N)$ in low dimension. The catch is that the pruning stops working as n grows — by roughly $n \approx 20$ the tree examines almost everything and is slower than brute force, for the reason given in §8.3.

- **Ball trees.** Split by enclosing balls rather than coordinate planes. They tolerate higher dimension better than k -d trees and work with any metric, not only Euclidean.
- **Triangle-inequality pruning.** If $d(\mathbf{q}, \mathbf{x})$ is known and $d(\mathbf{x}, \mathbf{y})$ was precomputed, then **M4** of the vector unit, §14.1, gives $d(\mathbf{q}, \mathbf{y}) \geq d(\mathbf{q}, \mathbf{x}) - d(\mathbf{x}, \mathbf{y})$, and if that lower bound already exceeds the current k -th best distance, $\mathbf{y}$ can be discarded without ever being measured. This is a metric axiom doing computational work, and Part 3 will use the same trick on k -means.

Approximate methods (ANN)

Above a few dozen dimensions, exact methods lose to brute force, and the practical answer is to stop insisting on exactness. **Approximate nearest neighbour** search returns neighbours that are *almost always* the true ones, thousands of times faster.

- **LSH** (locality-sensitive hashing) hashes vectors so that nearby points collide with high probability; a query examines only its own bucket.
- **HNSW** (hierarchical navigable small world) builds a layered proximity graph — a k -NN graph, §9.2, with long-range shortcuts — and answers a query by greedy descent. It is the current default in most systems.
- **IVF with product quantization** partitions the space into cells, searches only the nearest few, and stores each vector in compressed form so that millions fit in memory.

Remark. The trade is explicit and it is measured. $Recall@k$ is the fraction of the true k neighbours that a method actually returns, and ANN systems are tuned to a target such as 95% recall at some queries-per-second budget. Whether losing one neighbour in twenty matters depends entirely on the application — for retrieving passages to answer a question it is invisible; for a legal or medical audit trail it may not be acceptable.

10.3. What the industry uses

Tool	What it is
scikit-learn	KNeighborsClassifier. Its <code>algorithm="auto"</code> chooses between brute force, a k -d tree and a ball tree from N and n . The teaching and prototyping standard.
FAISS	Meta’s library for large-scale similarity search, and the reference implementation: IVF, product quantization, GPU support.
hnswlib, ScaNN, Annoy	Focused single-index ANN implementations, the latter two from Google and Spotify.
Vector databases	Pinecone, Weaviate, Milvus, Qdrant, and the pgvector extension to PostgreSQL. These are ANN indexes wrapped in a database: storage, filtering, updates and a query API.

Two things are worth noticing about that table. The first is that almost all of it is recent — the vector-database category barely existed before large language models made embedding-and-retrieving the standard way to give a model access to documents. The second is that every entry is an implementation of §6.3. The rule being computed is still: measure the distances, take the k smallest, and let them vote. Everything else is an index built to avoid measuring all N of them.

Looking ahead. Part 3 turns to the other problem. There will be no labels, so there is nothing to vote; the task is to find the groups in the data rather than to place a new point into known ones. The tools are the same — distances between vectors, and counting the cost of computing them — and one new object is needed, the mean of a set of vectors.